Analysis of Monolithic and Microservices Architecture

Design, Scalability, and Trade-offs

By: Chaitanya Deshpande

Made with GAMMA

INTRODUCTION

Two Architectural Paradigms

Monolithic Architecture

- Monolithic architecture is a traditional software design approach where the entire application is built as a single unified system.
- All functional components such as user interface, business logic, and data access are tightly integrated in one codebase.
- The application is developed, deployed, and scaled as one unit.
- Components share the same database and runtime environment.
- It is widely used in small to medium-scale applications due to its simplicity.

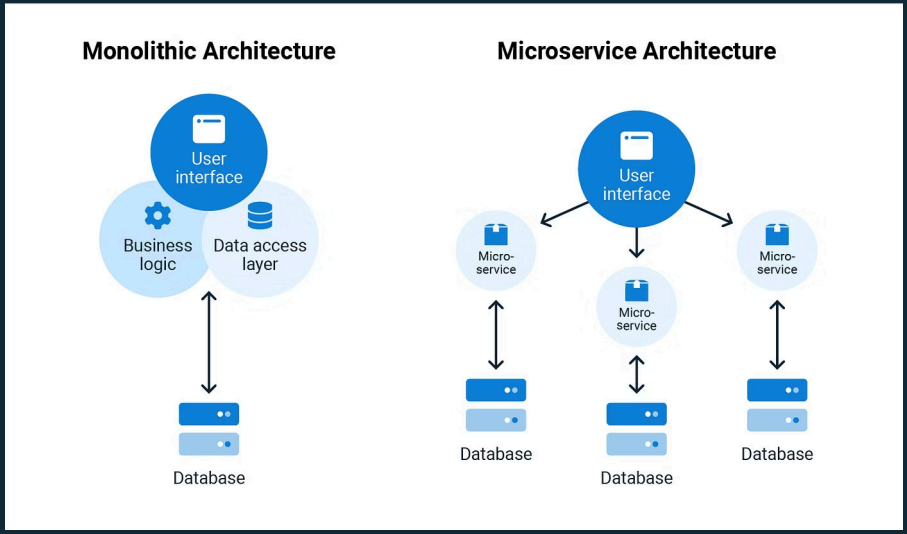
Microservices Architecture

- Microservices architecture is a modern approach where an application is divided into multiple small, independent services.
- Each service represents a specific business capability and runs in its own process.
- Services communicate with each other through APIs or messaging.
- Each microservice can be developed, deployed, and scaled independently.
- It is widely adopted in large-scale and cloud-native systems.

Made with GAMMA

ARCHITECTURE FLOW

Architecture and Working Flow



Monolithic Flow

- Represented as a single system boundary containing all application components.
- Internal modules (UI, business logic, data access) are modeled as tightly coupled components.
- Components interact through direct associations within the same deployment unit.
- A shared database is depicted as a common dependency for all modules.
- Emphasizes centralized control and in-process communication inside one application container.

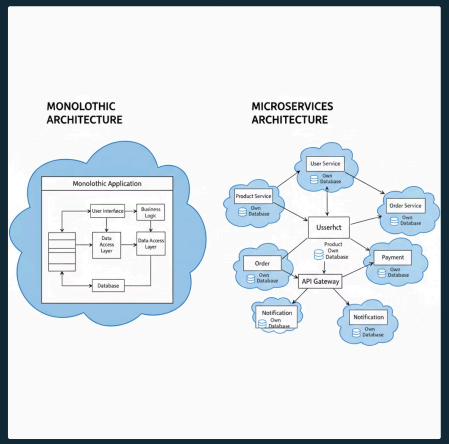
Microservices Flow

- Modeled as multiple independent service components, each with its own system boundary.
- Every service is associated with a dedicated datastore, indicating decentralized data ownership.
- External clients interact through an API Gateway or service interface component.
- Inter-service relationships are shown via interface-based or message-based communication links.
- Highlights distributed structure, loose coupling, and independent deployment of services.

Made with GAMMA

UML OVERVIEW

UML Diagram Comparison



Monolithic Architecture

- Represented as a single system boundary containing all application components.
- Internal modules (UI, business logic, data access) are modeled as tightly coupled components.
- Components interact through direct associations within the same deployment unit.
- A shared database is depicted as a common dependency for all modules.
- Emphasizes centralized control and in-process communication inside one application container.

Microservices Architecture

- Modeled as multiple independent service components, each with its own system boundary.
- Every service is associated with a dedicated datastore, indicating decentralized data ownership.
- External clients interact through an API Gateway or service interface component.
- Inter-service relationships are shown via interface-based or message-based communication links.
- Highlights distributed structure, loose coupling, and independent deployment of services.

Made with GAMMA

COMPARISON

Pros and Cons Analysis

Monolithic Architecture

Pros

- Simple architecture and straightforward development
- Easy testing and debugging within single codebase
- Lower initial deployment complexity
- Efficient in-process communication
- Ideal for small to medium applications

Cons

- Limited scalability—entire app scales together
- Tight coupling hinders maintenance
- Technology stack lock-in
- Large codebase reduces agility
- Single point of failure

Microservices Architecture

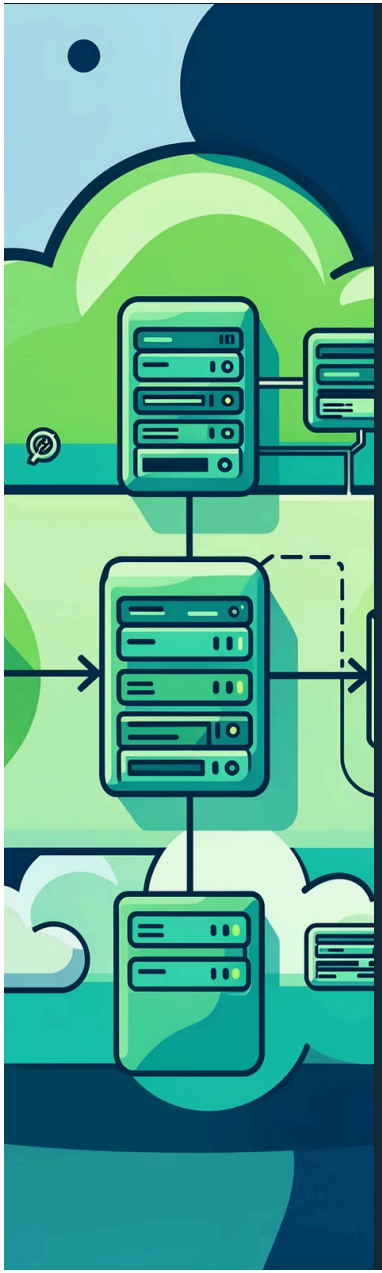
Pros

- Independent service scalability
- Loose coupling improves flexibility
- Technology heterogeneity enabled
- Faster feature delivery
- High resilience and fault isolation

Cons

- Higher architectural complexity
- Requires advanced DevOps and monitoring
- Network latency overhead
- Distributed data management challenges
- Complex testing across services

Made with GAMMA



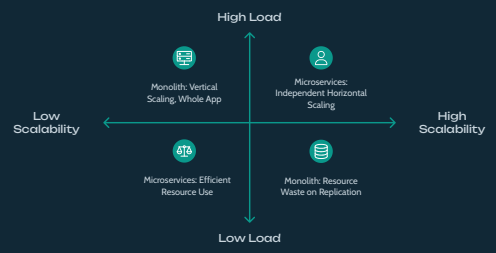
Scalability Analysis

Monolithic Scalability

- The whole application must be scaled together.
- If one module gets heavy traffic, entire system is replicated.
- Usually scaled by increasing server power or adding full copies.
- This wastes resources because all parts scale even if not needed.
- Scaling becomes difficult as the application grows large.

Microservices Scalability

- Only the required service is scaled based on demand.
- High-load services can scale without affecting others.
- New service instances can be added easily in cloud environments.
- Resources are used efficiently because scaling is targeted.
- Supports large-scale and high-traffic applications effectively.

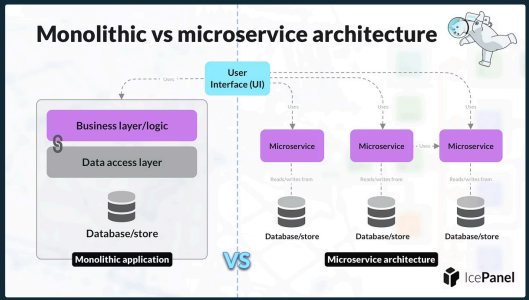


Microservices architecture enables granular, targeted scaling that optimizes infrastructure costs while maintaining performance under variable load conditions.

Made with GAMMA

MIGRATION STORY

Why Applications Move from Monolithic to Microservices



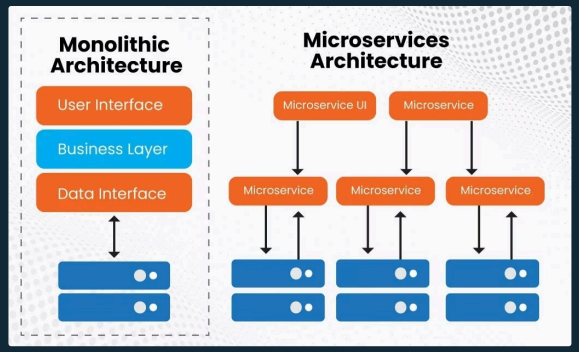
- In monolithic systems, all features are combined in one application.
- As the system grows, it becomes large and difficult to manage.
- The whole application must be scaled even if only one part needs it.
- Any small change requires redeploying the entire system.
- Microservices separate features into independent services for better scalability and maintenance.

Example: Netflix — Monolithic to Microservices

- Netflix initially started with a monolithic architecture where streaming, user accounts, recommendations, and billing were part of one system.
- As global users increased, streaming and recommendation features required much higher scalability than other modules.
- Scaling the entire monolithic system became inefficient and difficult to maintain.
- Netflix migrated to microservices by separating services like Streaming Service, User Service, Recommendation Service, and Billing Service.

Made with GAMMA

Market Analysis and Trade-offs



Market Trends

- Small and medium companies prefer monolithic for faster development and lower costs
- Startups begin with monolithic to build MVP products quickly
- Large enterprises adopt microservices for scalability and reliability
- Cloud computing and DevOps accelerate microservices adoption
- Modern platforms (Netflix, Amazon, Uber) leverage microservices at scale

Trade-off in Market

- Monolithic offers lower initial cost and simplicity, but limited long-term scalability.
- Microservices provide scalability and flexibility, but require higher infrastructure and expertise.
- Organizations choose architecture based on product size, team maturity, and growth expectations.
- Early-stage products favor monolithic, while scaling businesses transition to microservices.

Made with GAMMA

Conclusion

- Monolithic architecture provides simplicity, centralized control, and faster initial development.
- Microservices architecture enables scalability, flexibility, and independent service evolution.
- Monolithic systems are suitable for small to medium or stable applications.
- Microservices are better suited for large-scale, high-traffic, and continuously evolving systems.
- The choice of architecture depends on business size, growth expectations, and technical maturity.
- Industry trends show gradual migration from monolithic to microservices as systems scale.

Made with GAMMA

Thank You !!!

Questions and Discussion